

```
# -*- coding: utf-8 -*-
"""
25 (GitHub Actions
Ready)
=====
=====
🌟 v25 الجديد في:

NEW ميزات جديدة:
✅ وضع --once: مثالي لـ دورة واحدة ثم خروج (GitHub Actions)
✅ وضع --max-posts N: حد أقصى للمنشورات في كل
تشغيل
✅ وضع --verify-sources: فحص المصادر فقط
بدون نشر
✅ مصادر إخبارية عراقية جديدة (المجموع: 15 مصدر) 7
✅ GitHub Actions خطوط إضافية للتوافق مع
runners

🔗 المصادر الجديدة:
✅ (non14) نون نيوز - non14.net
✅ كتابات - kitabat.com
✅ شبكة أخبار العراق - aliraqnews.com
✅ المسلة - almasalah.com
✅ صوت العراق - sotaliraq.com
✅ Iraq Business News - iraq-
businessnews.com
✅ INA رابط احتياطي -
ina.iq/rss_feed.xml
```

💡 الفحفاظ عليها v24 تذكير بمميزات:

- ✓ python-bidi (عرض عربي صحيح)
- ✓ retry counter (لا فقدان أخبار)
- ✓ جلب متوازي (15 مصدر بالتوازي)
- ✓ font cache (تسريع توليد الصور)
- ✓ fast gradient (Image.linear_gradient)
- ✓ JPEG output (70 % حجم أقل بـ)
- ✓ logging مع مستويات
- ✓ معالجة SIGTERM/SIGINT
- ✓ hash-based dedup
- ✓ urljoin للروابط النسبية

📁 ملف DB: /tmp/news_cache_v25.db (قابل للتعديل)
عبر NEWS_DB_PATH)

🚀 طرق التشغيل:

```
python
publisher_v25.py # حلقة لا
# نهائية (VPS/PythonAnywhere)
python publisher_v25.py --
once # دورة واحدة (GitHub
Actions)
python publisher_v25.py --once --max-
posts 5 # حد أقصى 5 منشورات
python publisher_v25.py --verify-
sources # فحص المصادر فقط
"""
```

```
import os
import sys
import io
import re
import time
import signal
```

```

import hashlib
import sqlite3
import logging
import argparse
import traceback
from datetime import datetime
from concurrent.futures import
ThreadPoolExecutor, as_completed
from typing import Optional, Dict, List,
Tuple
from urllib.parse import urljoin

#
=====
=====
# أولاً (قبل أي شيء آخر) إعداد Logging
#
=====
=====
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s [%(levelname)s] %(
message)s',
    datefmt='%H:%M:%S',
    handlers=
[logging.StreamHandler(sys.stdout)],
)
logger = logging.getLogger('publisher')

# قمع رسائل المكتبات الخارجية المزعجة
logging.getLogger('urllib3').setLevel(loggi
ng.WARNING)
logging.getLogger('PIL').setLevel(logging.W

```

```

ARNING)
logging.getLogger('requests').setLevel(logging.WARNING)

#
=====
=====
# fallback استيراد المكتبات الخارجية مع معالجة
#
=====
=====
try:
    import requests
    import feedparser
    from bs4 import BeautifulSoup
    from PIL import Image, ImageDraw,
    ImageFont, ImageOps, ImageEnhance
except ImportError as e:
    logger.error(f"✗ مكتبة أساسية مفقودة: {e}")
    logger.error(" شغل: pip install -r requirements.txt")
    sys.exit(1)

# v24 معالجة النص العربي - الإضافة الحرجة في
try:
    import arabic_reshaper
    from bidi.algorithm import get_display
    _HAS_RTL = True
except ImportError as e:
    logger.warning(f"⚠ مكتبات RTL مفقودة: {e}")
    logger.warning(" النص العربي قد يظهر معكوساً !على الصور")

```

```

        logger.warning(" شغل: pip install
arabic-reshaper python-bidi")
        _HAS_RTL = False

def ar(text: Optional[str]) -> str:
    """(reshape +
bidi).

    فقط، مما يسبب عرض reshape كان يستخدم v23
    الكلمات معكوسة على الصور
    v24 من python-bidi من get_display يضيف
    لترتيب صحيح للنص
    """
    if not text:
        return ""
    if not _HAS_RTL:
        return str(text)
    try:
        reshaped =
arabic_reshaper.reshape(str(text))
        return get_display(reshaped)
    except Exception as e:
        logger.debug(f"خطأ في معالجة النص العربي:
{e}")
        return str(text)

#
=====
=====
# Cache (تحسين أداء كبير)
#
=====

```

```
=====
FONT_PATHS: Tuple[str, ...] = (
    # Ubuntu/Debian - الخطوط القياسية على
    Actions runners
    "/usr/share/fonts/opentype/fonts-hosny-
    amiri/Amiri-Bold.ttf",
    "/usr/share/fonts/truetype/amiri/Amiri-
    Bold.ttf",

    "/usr/share/fonts/truetype/amiri/AmiriQuran
    -Regular.ttf",

    "/usr/share/fonts/truetype/noto/NotoNaskhAr
    abic-Bold.ttf",

    "/usr/share/fonts/truetype/noto/NotoNaskhAr
    abicUI-Bold.ttf",

    "/usr/share/fonts/truetype/noto/NotoSansAra
    bic-Bold.ttf",

    "/usr/share/fonts/truetype/noto/NotoKufiAra
    bic-Bold.ttf",

    "/usr/share/fonts/truetype/noto/NotoSans-
    Bold.ttf",

    "/usr/share/fonts/truetype/dejavu/DejaVuSan
    s-Bold.ttf",
)

_font_cache: Dict[int, ImageFont.ImageFont]
= {}
_resolved_font_path: Optional[str] = None
```

```

_font_path_searched: bool = False

def _resolve_font_path() -> Optional[str]:
    """
    البحث عن أول خط متاح (مرة واحدة فقط طول
    العمر)."""
    global _resolved_font_path,
    _font_path_searched
    if _font_path_searched:
        return _resolved_font_path
    _font_path_searched = True
    for path in FONT_PATHS:
        if os.path.exists(path):
            _resolved_font_path = path
            logger.info(f"✍ الخط المستخدم:
{path}")
            return path
    logger.warning("⚠ لم يُعثر على خط عربي، الخط
    الافتراضي قد لا يدعم العربية")
    return None

def get_font(size: int) ->
    ImageFont.ImageFont:
    """
    بدل تحميله في cache جلب خط بحجم محدد مع
    (كل صورة)."""
    if size in _font_cache:
        return _font_cache[size]

    path = _resolve_font_path()
    try:
        font = ImageFont.truetype(path,
    size) if path else ImageFont.load_default()
    except (OSError, IOError) as e:

```

```

        logger.warning(f"خطأ في تحميل الخط بحجم {size}: {e}")
        font = ImageFont.load_default()

        _font_cache[size] = font
        return font

#
=====
=====
# import مرة واحدة عند ال (قراءة متغيرات البيئة)
#
=====
=====
def _load_token(env_name: str) -> str:
    """قراءة توكن الصفحة من متغير البيئة"""

    v23: يدعم صيغتين للتوافق مع
    - 'page_id|token' (الصيغة القديمة المركبة)
    - 'token' (الصيغة الجديدة المباشرة - أبسط)
    """
    val = os.environ.get(env_name,
    '').strip()
    if not val:
        return ''
    if '|' in val:
        parts = val.split('|', 1)
        return parts[1].strip() if
len(parts) > 1 else ''
    return val

#

```



```

=====
=====
# الأرقام والشهور العربية (للتاريخ في التصميم)
#
=====
=====
ARABIC_NUMERALS =
str.maketrans('0123456789', '٠١٢٣٤٥٦٧٨٩')
ARABIC_MONTHS = {
    1: 'يناير', 2: 'فبراير', 3: 'مارس', 4: 'أبريل',
    5: 'مايو', 6: 'يونيو', 7: 'يوليو', 8: 'أغسطس',
    9: 'سبتمبر', 10: 'أكتوبر', 11: 'نوفمبر', 12: 'ديسمبر'
}

def format_arabic_date(dt:
Optional[datetime] = None) -> str:
    """تنسيق التاريخ بالعربي. مثال: ١٦ يونيو ٢٠٢٦"""
    if dt is None:
        dt = datetime.now()
    day =
str(dt.day).translate(ARABIC_NUMERALS)
    month = ARABIC_MONTHS.get(dt.month, '')
    year =
str(dt.year).translate(ARABIC_NUMERALS)
    return f"{day} {month} {year}"

#
=====
=====

```

```

# إعدادات الصفحات (ألوان احترافية + شعار لكل صفحة)
#
=====
=====
PAGES_CONFIG: Dict[str, Dict] = {
    'salssal': {
        'page_id': '1104346172760947',
        'name': 'صلصال',
        'logo_letter': 'ص',
        'token':
_load_token('PAGE_SALSSAL'),
        'bar_grad_start': (210, 175, 100),
        'bar_grad_end': (140, 100, 45),
        'bar_text': (255, 255, 255),
        'border1': (220, 185, 110),
        'border2': (160, 120, 55),
        'glow': (255, 240, 180),
        'overlay_color': (20, 10, 0),
        'deco_color': (255, 220, 120),
        'accent': (200, 50, 50),
    },
    'chai': {
        'page_id': '1078693568663658',
        'name': 'چاي سادة',
        'logo_letter': 'چ',
        'token': _load_token('PAGE_CHAI'),
        'bar_grad_start': (35, 22, 8),
        'bar_grad_end': (15, 8, 2),
        'bar_text': (212, 175, 55),
        'border1': (212, 175, 55),
        'border2': (140, 105, 25),
        'glow': (255, 215, 80),
        'overlay_color': (10, 5, 0),
        'deco_color': (212, 175, 55),
    },
}

```

```

        'accent':          (220,  40,  40),
    },
    'taboga': {
        'page_id': '1063874040148711',
        'name': 'طابوكة',
        'logo_letter': 'ط',
        'token':
_load_token('PAGE_TABOGA'),
        'bar_grad_start': (  5,   5,   5),
        'bar_grad_end':   (  0,   0,   0),
        'bar_text':       (212, 175,  55),
        'border1':        (212, 175,  55),
        'border2':        (140, 105,  25),
        'glow':           (255, 215,  80),
        'overlay_color':  (  0,   0,   0),
        'deco_color':     (212, 175,  55),
        'accent':         (220,  40,  40),
    },
    'tein': {
        'page_id': '1094102397116855',
        'name': 'طين',
        'logo_letter': 'ط',
        'token': _load_token('PAGE_TEIN'),
        'bar_grad_start': (230, 215, 185),
        'bar_grad_end':   (200, 180, 145),
        'bar_text':       ( 90,  55,  20),
        'border1':        (180, 145,  90),
        'border2':        (120,  90,  45),
        'glow':           (200, 160,  80),
        'overlay_color':  ( 60,  35,  10),
        'deco_color':     (150, 110,  55),
        'accent':         (180,  40,  40),
    },
}

```

```
#
=====
=====
# مصادر الأخبار والكلمات المفتاحية
#
=====
=====
NEWS_SOURCES: Tuple[str, ...] = (
    # ===== عراقية رئيسية =====

    'https://www.alsumaria.tv/rss',          #
    السومرية

    'https://www.shafaq.com/ar/rss.xml',      #
    شفق نيوز

    'https://www.rudaw.net/arabic/rss',       #
    رووداو

    'https://www.ina.iq/rss.xml',            #
    رابطة أساسي INA

    'https://www.mawazin.net/rss',           #
    موازين نيوز

    # ===== مضافة في مصادر عراقية جديدة =====

    'https://www.ina.iq/rss_feed.xml',        #
    رابطة احتياطي INA

    'http://non14.net/services/rss',          #
```

نون نيوز

'https://kitabab.com/feed/', #
كتابات

'https://aliraqnews.com/feed/', #
شبكة أخبار العراق

'https://almasalah.com/rss/', #
المسلة

'https://www.sotaliraq.com/feed/', #
صوت العراق

'https://www.iraq-
businessnews.com/feed/', # Iraq Business
News

===== مصادر إقليمية (من v23) =====

'https://www.aljazeera.net/rss/all.xml', #
الجزيرة

'https://feeds.bbc.co.uk/arabic/rss.xml',
BBC العربية

'https://arabic.rt.com/rss/', #
RT العربية

===== مضافة في مصادر إقليمية جديدة (v26)
=====

'https://www.alarabiya.net/.mrss/ar.xml',
العربية (الحدث)

```

'https://www.alarabiya.net/.mrss/ar/aswaq.xml', # العربية أسواق

'https://www.skynewsarabia.com/web/feeds/rss/middle-east.xml', # سكاي نيوز

'https://www.france24.com/ar/rss',
# France 24 عربي
    'https://rss.dw.com/rdf/rss-ar-all', # DW عربي
    'https://al-ain.com/rss', # العين الإخبارية
)

IRAQ_KEYWORDS: Tuple[str, ...] = (
    'العراق', 'عراق', 'بغداد', 'البصرة', 'الموصل', 'كركوك',
    'أربيل', 'السليمانية', 'النجف', 'كربلاء', 'الكاظمي',
    'السوداني', 'الحكومة العراقية', 'البرلمان العراقي',
    'الجيش العراقي', 'الحشد الشعبي', 'الكرد', 'العراقي',
    'العراقية', 'العراقيين', 'دينار', 'نפט العراق'
)

#
=====
=====
# Headers (User-Agent) موحدة (كامل وحديث)
#
=====
=====
USER_AGENT = (
    'Mozilla/5.0 (Windows NT 10.0; Win64;

```

```

x64) '
    'AppleWebKit/537.36 (KHTML, like Gecko)
    ,
    'Chrome/120.0.0.0 Safari/537.36'
)
REQUEST_HEADERS = {'User-Agent':
USER_AGENT}
IMAGE_HEADERS = {
    'User-Agent': USER_AGENT,
    'Accept':
    'image/webp,image/apng,image/*,*/*;q=0.8',
    'Referer': 'https://www.google.com/',
}

#
=====
=====
# الإعدادات العامة
#
=====
=====
DB_PATH = os.environ.get('NEWS_DB_PATH',
'/tmp/news_cache_v25.db')
MAX_RETRIES_PER_NEWS = 3 #
محاولات قبل التخلي عن الخبر
MAX_DB_INIT_ATTEMPTS = 5 #
محاولات تهيئة قاعدة البيانات
REQUIRE_IMAGE = True #
نشر الأخبار التي لها صورة فقط: v26
NEWS_CLEANUP_AFTER_DAYS = 7 # حذف
أيام (زدنا من 3 إلى 7) X الأخبار الأقدم من
FETCH_INTERVAL_CYCLES = 3 # جلب
دورات (30=10×3 ثانية) X أخبار كل

```

```

CLEANUP_INTERVAL_CYCLES = 120 #
دورة (20=10×120 دقيقة) X تنظيف كل
LOOP_DELAY_SECONDS = 10 #
انتظار بين الدورات
REQUEST_TIMEOUT = 8 #
الصور/RSS لطلبات timeout
FB_POST_TIMEOUT = 30 #
للنشر على فيسبوك timeout
IMAGE_QUALITY = 88 # جودة
JPEG (88 = جيد توازن)

#
=====
=====
# الإيقاف اللطيف (graceful shutdown)
#
=====
=====
_shutdown_requested = False

def _signal_handler(signum, _frame):
    """ Docker / GitHub
    معالج إشارات الإيقاف لحاويات
    Actions / k8s."""
    global _shutdown_requested
    try:
        sig_name =
        signal.Signals(signum).name
    except (ValueError, AttributeError):
        sig_name = str(signum)
    logger.info(f"⚠️ استلام إشارة {sig_name}،
    الإيقاف اللطيف بعد الدورة الحالية")
    _shutdown_requested = True

```



```

#
=====
=====
# قاعدة البيانات
#
=====
=====
def init_db() -> bool:
    """عند النجاح True تهيئة قاعدة البيانات. يُرجع"""
    try:
        conn = sqlite3.connect(DB_PATH)
        c = conn.cursor()

        c.execute('''
            CREATE TABLE IF NOT EXISTS news
            (
                id INTEGER PRIMARY KEY
            AUTOINCREMENT,
                title_hash TEXT UNIQUE NOT
            NULL,
                title TEXT NOT NULL,
                url TEXT,
                image_url TEXT,
                retry_count INTEGER DEFAULT
            0,
                saved_at TIMESTAMP DEFAULT
            CURRENT_TIMESTAMP,
                posted INTEGER DEFAULT 0,
                posted_at TIMESTAMP
            )
        ''')

```

```

# فهرس لتسريع الاستعلامات
c.execute('CREATE INDEX IF NOT
EXISTS idx_posted ON news(posted,
retry_count)')
c.execute('CREATE INDEX IF NOT
EXISTS idx_saved_at ON news(saved_at)')

conn.commit()
conn.close()
logger.info(f"✅ قاعدة البيانات جاهزة:
{DB_PATH}")
return True
except sqlite3.Error as e:
    logger.error(f"❌ خطأ SQLite في
init_db: {e}")
    return False
except Exception as e:
    logger.error(f"❌ خطأ عام في
init_db: {e}")
    return False

def _title_hash(title: str) -> str:
    """الذكي dedup للعنوان لا hash حساب"""

    hash: يُطَبِّع العنوان قبل الـ
    - يزيل علامات الترقيم
    - يدمج المسافات
    - lowercase يحول لـ

    هذا يجعل العناوين المتشابهة (مع اختلاف ترقيم/مسافات)
    تُعتبر نفس الخبر.
    """
    if not title:

```

```

        return ''
# v26: تطبيع عربي محسن لكشف أفضل للتكرار
normalized = title

# 1. توحيد الألف (أ، إ، آ) إلى ا
normalized = re.sub(r'[إآ]', 'ا',
normalized)
# 2. توحيد الياء (ى → ي)
normalized = normalized.replace('ى', 'ي')
# 3. توحيد التاء المربوطة (ة → ه)
normalized = normalized.replace('ة', 'ه')
# 4. إزالة التشكيل (الحركات)
normalized = re.sub(r'[\u064B-\u0652\u0670\u0640]', '', normalized)
# 5. احتفظ بحروف، أرقام، مسافات فقط
normalized = re.sub(r'^\w\s\u0600-\u06FF]', ' ', normalized)
# 6. دمج المسافات وتصغير الحروف
normalized = re.sub(r'\s+', ' ', normalized).strip().lower()

if not normalized or len(normalized) <
10:
    return ''
return
hashlib.md5(normalized.encode('utf-
8')).hexdigest()

def save_news(title: str, url: str,
image_url: str = '') -> bool:
    """إذا True يُرجع hash. ب dedup حفظ خبر مع"""

```

تم الإدراج (خبر جديد)

```
لا يحفظ الأخبار بدون صورة: v26 منذ
(REQUIRE_IMAGE=True).
"""
if not title or len(title) < 10:
    return False
# v26: نتجاهل الأخبار بدون صورة
if REQUIRE_IMAGE and not image_url:
    return False
h = _title_hash(title)
if not h:
    return False
try:
    conn = sqlite3.connect(DB_PATH)
    c = conn.cursor()
    c.execute(
        'INSERT OR IGNORE INTO news
(title_hash, title, url, image_url) VALUES
(?, ?, ?, ?)',
        (h, title, url, image_url)
    )
    inserted = c.rowcount > 0
    conn.commit()
    conn.close()
    return inserted
except sqlite3.Error as e:
    logger.debug(f"خطأ في save_news:
{e}")
    return False

def get_unposted() -> Optional[Dict]:
    """ جلب أول خبر لم يُنشر (ولم يتجاوز عدد
```

(المحاولات).

```
لا يجلب الأخبار بدون صورة: v26 منذ  
(REQUIRE_IMAGE=True).  
""  
try:  
    conn = sqlite3.connect(DB_PATH)  
    c = conn.cursor()  
    if REQUIRE_IMAGE:  
        c.execute('''  
            SELECT id, title, url,  
image_url, retry_count  
            FROM news  
            WHERE posted=0 AND  
retry_count < ? AND image_url IS NOT NULL  
AND image_url != ''  
            ORDER BY id ASC  
            LIMIT 1  
            ''', (MAX_RETRIES_PER_NEWS,))  
    else:  
        c.execute('''  
            SELECT id, title, url,  
image_url, retry_count  
            FROM news  
            WHERE posted=0 AND  
retry_count < ?  
            ORDER BY id ASC  
            LIMIT 1  
            ''', (MAX_RETRIES_PER_NEWS,))  
    row = c.fetchone()  
    conn.close()  
    if row:  
        return {  
            'id': row[0],
```

```

        'title': row[1],
        'url': row[2] or '',
        'image_url': row[3] or '',
        'retry_count': row[4],
    }
except sqlite3.Error as e:
    logger.warning(f"خطأ في {e}")
get_unposted: {e}")
return None

def mark_posted(news_id: int) -> None:
    """تعليم الخبر كمنشور."""
    try:
        conn = sqlite3.connect(DB_PATH)
        c = conn.cursor()
        c.execute(
            'UPDATE news SET posted=1,
            posted_at=CURRENT_TIMESTAMP WHERE id=?',
            (news_id,)
        )
        conn.commit()
        conn.close()
    except sqlite3.Error as e:
        logger.warning(f"خطأ في {e}")
mark_posted({news_id}): {e}")

def increment_retry(news_id: int) -> int:
    """زيادة عدد المحاولات. يُرجع القيمة الجديدة (للقرار: إعادة محاولة أم تخطي)"""
    try:
        conn = sqlite3.connect(DB_PATH)
        c = conn.cursor()

```

```

        c.execute('UPDATE news SET
retry_count = retry_count + 1 WHERE id=?',
(news_id,))

        c.execute('SELECT retry_count FROM
news WHERE id=?', (news_id,))
        row = c.fetchone()
        conn.commit()
        conn.close()
        return row[0] if row else 0
    except sqlite3.Error as e:
        logger.warning(f"خطأ في
increment_retry({news_id}): {e}")
        return 0

def cleanup_old() -> int:
    """حذف الأخبار القديمة. يُرجع عدد المحذوفات."""
    try:
        conn = sqlite3.connect(DB_PATH)
        c = conn.cursor()
        c.execute(
            "DELETE FROM news WHERE
saved_at < datetime('now', ?)",
            (f'-{NEWS_CLEANUP_AFTER_DAYS}
days',)
        )
        deleted = c.rowcount
        conn.commit()
        conn.close()
        return deleted
    except sqlite3.Error as e:
        logger.warning(f"خطأ في
cleanup_old: {e}")
        return 0

```

```

#
=====
=====
# (متوازي الآن - تسريع 8) جلب الأخبار
#
=====
=====
def is_iraq_news(text: str) -> bool:
    """التحقق من أن الخبر يتعلق بالعراق."""
    if not text:
        return False
    return any(kw in text for kw in
IRAQ_KEYWORDS)

def _normalize_image_url(image_url: str,
source_url: str) -> str:
    """إصلاح بق الصورة النسبي إلى مطلق URL تحويل (v23 في
في v23)."""
    if not image_url:
        return ''
    image_url = image_url.strip()
    if image_url.startswith(('http://',
'https://')):
        return image_url
    if not source_url:
        return ''
    try:
        return urljoin(source_url,
image_url)
    except (ValueError, TypeError):
        return ''

```



```

def extract_image(entry, source_url: str =
'') -> str:
    """URL مع تطبيع RSS استخراج صورة من خبر"""
    try:
        # من media_content
        if hasattr(entry, 'media_content')
and entry.media_content:
            for m in entry.media_content:
                url = m.get('url', '')
                if url:
                    return
            _normalize_image_url(url, source_url)

        # من media_thumbnail
        if hasattr(entry,
'media_thumbnail') and
entry.media_thumbnail:
            url =
entry.media_thumbnail[0].get('url', '')
            if url:
                return
            _normalize_image_url(url, source_url)

        # من enclosures
        if hasattr(entry, 'enclosures') and
entry.enclosures:
            for enc in entry.enclosures:
                if 'image' in
enc.get('type', ''):
                    url = enc.get('href',
'') or enc.get('url', '')
                    if url:

```

```

        return
    _normalize_image_url(url, source_url)

    # من summary (HTML)
    if hasattr(entry, 'summary') and
entry.summary:
        soup =
BeautifulSoup(entry.summary, 'html.parser')
        img = soup.find('img')
        if img and img.get('src'):
            return
_normalize_image_url(img['src'],
source_url)
    except Exception as e:
        logger.debug(f"خطأ في
extract_image: {e}")
    return ''

def _fetch_from_source(source: str) ->
List[Dict]:
    """جلب الأخبار من مصدر واحد (يُنْفَذ بالتوازي)"""
    news_items: List[Dict] = []
    try:
        resp = requests.get(source,
timeout=REQUEST_TIMEOUT,
headers=REQUEST_HEADERS)
        if resp.status_code != 200:
            logger.debug(f"{source} رد بـ
{resp.status_code}")
        return news_items

    feed =
feedparser.parse(resp.content)

```

```

        for entry in feed.entries[:15]:
            try:
                title = entry.get('title',
''.strip()
                url = entry.get('link', '')
                if not title or len(title)
< 10:
                    continue
                summary =
entry.get('summary', '')
                if not is_iraq_news(title +
' ' + summary):
                    continue
                image_url =
extract_image(entry, source)
                news_items.append({
                    'title': title,
                    'url': url,
                    'image_url': image_url,
                })
            except Exception as e:
                logger.debug(f"خطأ في entry
من {source}: {e}")
                continue
            except requests.exceptions.Timeout:
                logger.debug(f"timeout: {source}")
            except
requests.exceptions.RequestException as e:
                logger.debug(f"خطأ شبكة {source}:
{e}")
            except Exception as e:
                logger.debug(f"خطأ غير متوقع {source}:
{e}")

```

```

        return news_items

def fetch_news() -> List[Dict]:
    """جلب الأخبار من جميع المصادر بالتوازي."""
    all_news: List[Dict] = []
    overall_timeout = REQUEST_TIMEOUT * 2 +
5

    with
ThreadPoolExecutor(max_workers=len(NEWS_SOU
RCES)) as executor:
        futures =
{executor.submit(_fetch_from_source, src):
src for src in NEWS_SOURCES}
        try:
            for future in
as_completed(futures,
timeout=overall_timeout):
                try:
                    items =
future.result(timeout=1)
                    all_news.extend(items)
                except Exception as e:
                    logger.debug(f"فشل
{futures[future]}: {e}")
            except TimeoutError:
                logger.warning(f"⌚ timeout عام
في fetch_news ({overall_timeout}s)")
            except Exception as e:
                logger.warning(f"خطأ غير متوقع في
fetch_news: {e}")

    return all_news

```

```

#
=====
=====
# توليد الصور
#
=====
=====
def download_image(url: str) ->
Optional[Image.Image]:
    """URL تحميل صورة من"""
    if not url:
        return None
    try:
        resp = requests.get(url,
        timeout=REQUEST_TIMEOUT,
        headers=IMAGE_HEADERS, stream=True)
        if resp.status_code == 200:
            img =
Image.open(io.BytesIO(resp.content))
            return img.convert('RGB')
        except
requests.exceptions.RequestException as e:
            logger.debug(f"فشل تحميل {url}:
{e}")
        except (OSError, IOError) as e:
            logger.debug(f"خطأ معالجة الصورة {url}:
{e}")
        except Exception as e:
            logger.debug(f"خطأ غير متوقع للصورة
{url}: {e}")
    return None

```

```

def wrap_text(text: str, font, max_width:
int, draw) -> List[str]:
    """تقسيم النص إلى أسطر تناسب العرض"""
    words = text.split()
    lines: List[str] = []
    current: List[str] = []
    for word in words:
        test = ' '.join(current + [word])
        bbox = draw.textbbox((0, 0), test,
font=font)
        if bbox[2] - bbox[0] <= max_width:
            current.append(word)
        else:
            if current:
                lines.append('
'.join(current))
                current = [word]
            if current:
                lines.append(' '.join(current))
    return lines

```

```

def fast_gradient(
    width: int,
    height: int,
    start_color: Tuple[int, int, int],
    end_color: Tuple[int, int, int],
    vertical: bool = True
) -> Image.Image:
    """رسم تدرج لوني سريع"""

```

يرسم سطرًا سطرًا Python loop كان يستخدم v23
(بطيء جداً).

```

v24 يستخدم Image.linear_gradient +
ImageOps.colorize - داخلي C كله بكود .
x.تسريع تقريباً 20
"""

# linear_gradient يولد grayscale
256x256: أسود فوق، أبيض تحت
if vertical:
    grad =
Image.linear_gradient('L').resize((width,
height))
else:
    # تدوير 90° عكس عقارب الساعة: أسود يصير
    على اليسار
    grad =
Image.linear_gradient('L').rotate(90,
expand=True).resize((width, height))
    # colorize: أسود → start_color، أبيض →
end_color
    return ImageOps.colorize(grad,
start_color, end_color)

def _draw_breaking_news_badge(draw, x: int,
y: int, font, accent_color: tuple) -> int:
    """
    .تعيد عرض الشارة عند الانتهاء
    """
    text_ar = ar("خبر عاجل")
    padding_x, padding_y = 22, 10
    bbox = draw.textbbox((0, 0), text_ar,
font=font)
    tw = bbox[2] - bbox[0]
    th = bbox[3] - bbox[1]

```

```

        w = tw + padding_x * 2 + 30 # 30 للدائرة الحمراء
        h = th + padding_y * 2 + 4

        # خلفية الشارة - مستطيل مدور باللون الأحمر
        try:
            draw.rounded_rectangle(
                [x, y, x + w, y + h],
                radius=h // 2,
                fill=accent_color,
                outline=(255, 255, 255),
                width=2,
            )
        except AttributeError:
            # PIL قديم بدون rounded_rectangle
            draw.rectangle(
                [x, y, x + w, y + h],
                fill=accent_color, outline=
(255, 255, 255), width=2,
            )

        # دائرة بيضاء (بث مباشر)
        dot_size = 12
        dot_x = x + w - padding_x - dot_size //
2 - 4
        dot_y = y + h // 2
        draw.ellipse(
            [dot_x - dot_size // 2, dot_y -
dot_size // 2,
            dot_x + dot_size // 2, dot_y +
dot_size // 2],
            fill=(255, 255, 255),
        )

```



```

        inner = 5
        draw.ellipse(
            [dot_x - inner // 2, dot_y - inner
            // 2,
            dot_x + inner // 2, dot_y + inner
            // 2],
            fill=accent_color,
        )

        # النص
        text_x = x + padding_x - 5
        text_y = y + (h - th) // 2 - 4
        draw.text((text_x, text_y), text_ar,
        font=font, fill=(255, 255, 255))

    return w

```

```

def _draw_page_logo(draw, cx: int, cy: int,
radius: int, page_config: Dict, font) ->
None:
    """رسم شعار الصفحة (دائرة بحرف أول)"""
    p = page_config
    # الإطار الخارجي
    draw.ellipse(
        [cx - radius - 3, cy - radius - 3,
        cx + radius + 3, cy + radius + 3],
        outline=p['deco_color'],
        width=3,
    )
    # الدائرة الداخلية (الخلفية)
    draw.ellipse(
        [cx - radius, cy - radius, cx +
        radius, cy + radius],

```

```

        fill=p['bar_grad_start'],
        outline=p['border1'],
        width=2,
    )

    # حرف الشعار
    letter_ar = ar(p.get('logo_letter',
p['name'][0] if p.get('name') else ''))
    bbox = draw.textbbox((0, 0), letter_ar,
font=font)
    tw = bbox[2] - bbox[0]
    th = bbox[3] - bbox[1]

    # ظل دقيق
    for dx, dy in [(-1, -1), (1, 1)]:
        draw.text(
            (cx - tw // 2 + dx, cy - th //
2 + dy - 4),
            letter_ar, font=font,
fill=p['glow'],
        )

    # الحرف الرئيسي
    draw.text(
        (cx - tw // 2, cy - th // 2 - 4),
        letter_ar, font=font,
fill=p['bar_text'],
    )

def _draw_calendar_icon(draw, x: int, y:
int, size: int, color: tuple) -> None:
    """رسم أيقونة تقويم بسيطة"""
    try:
        draw.rounded_rectangle(

```

```

        [x, y + 4, x + size, y + size],
        radius=3, outline=color,
width=2,
    )
except AttributeError:
    draw.rectangle(
        [x, y + 4, x + size, y + size],
        outline=color, width=2,
    )
# خط علوي يفصل العنوان عن الأيام
draw.line(
    [(x, y + 12), (x + size, y + 12)],
    fill=color, width=2,
)
# الحلقتان العلويتان
draw.line([(x + 7, y), (x + 7, y + 8)],
fill=color, width=2)
draw.line([(x + size - 7, y), (x + size
- 7, y + 8)], fill=color, width=2)

def create_post_image(
    title: str,
    image_url: str,
    page_config: Dict
) -> Optional[Image.Image]:
    """إنشاء صورة المنشور بالتصميم الاحترافي الجديد"""
    (v26).

```

التصميم الجديد:

- شارة 'خبر عاجل' حمراء في الزاوية العلوية اليسرى
- شعار الصفحة (دائرة بالحرف الأول) في الزاوية العلوية اليمنى
- عنوان الخبر بخط واضح وظل احترافي

```

- شريط سفلي: تاريخ النشر + اسم الصفحة
"""
try:
    p = page_config
    W, H = 1200, 850
    BAR = 100 # شريط أكبر للأناقة
    PAD = 22

    canvas = Image.new('RGB', (W, H),
(0, 0, 0))

    # ===== الشريطان العلوي والسفلي =====
    top_bar = fast_gradient(W, BAR,
p['bar_grad_start'], p['bar_grad_end'],
vertical=True)
    canvas.paste(top_bar, (0, 0))
    bottom_bar = fast_gradient(W, BAR,
p['bar_grad_end'], p['bar_grad_start'],
vertical=True)
    canvas.paste(bottom_bar, (0, H -
BAR))

    draw = ImageDraw.Draw(canvas)

    # ===== الحدود الخارجية =====
    draw.rectangle([0, 0, W - 1, H -
1], outline=p['border1'], width=5)
    inner = 14
    draw.rectangle([inner, inner, W -
inner, H - inner], outline=p['border2'],
width=2)

    # ===== صورة الخبر داخل الإطار =====
    ix = PAD

```

```

iy = BAR + PAD // 2
iw = W - PAD * 2
ih = H - BAR * 2 - PAD

news_image =
download_image(image_url)
if news_image:
    news_resized =
news_image.resize((iw, ih), Image.LANCZOS)
    news_resized =
ImageEnhance.Contrast(news_resized).enhance
(1.1)
else:
    # خلفية احتياطية احترافية
    news_resized = fast_gradient(
        iw, ih, p['bar_grad_end'],
p['overlay_color'], vertical=True
    )
    bg_draw =
ImageDraw.Draw(news_resized)
    dc = p['deco_color']
    for xi in range(0, iw, 80):
        for yi in range(0, ih, 80):
            bg_draw.ellipse([xi -
2, yi - 2, xi + 2, yi + 2], fill=dc)
            lc = tuple(max(0, c - 20) for c
in p['bar_grad_start'])
            for xi in range(-ih, iw, 60):
                bg_draw.line([(xi, 0), (xi
+ ih, ih)], fill=lc, width=1)
            for xi in range(0, iw + ih,
60):
                bg_draw.line([(xi, 0), (xi
- ih, ih)], fill=lc, width=1)

```

```

        canvas.paste(news_resized, (ix,
iy))

        draw = ImageDraw.Draw(canvas)

        # ===== حدود حول الصورة =====
        draw.rectangle(
            [ix - 2, iy - 2, ix + iw + 2,
iy + ih + 2],
            outline=p['border2'], width=2
        )

        # ===== تعتييم تدريجي للنص =====
        overlay = Image.new('RGBA', (iw,
ih), (0, 0, 0, 0))
        ov_draw = ImageDraw.Draw(overlay)
        overlay_h = ih // 2
        for i in range(overlay_h):
            alpha = int(200 * (i /
overlay_h))
            ov_draw.line(
                [(0, ih - overlay_h + i),
(iw, ih - overlay_h + i)],
                fill=(*p['overlay_color'],
alpha)
            )

            canvas_rgba =
canvas.convert('RGBA')
            overlay_full = Image.new('RGBA',
(W, H), (0, 0, 0, 0))
            overlay_full.paste(overlay, (ix,
iy))

            canvas_rgba =
Image.alpha_composite(canvas_rgba,

```

```

overlay_full)
    canvas = canvas_rgba.convert('RGB')
    draw = ImageDraw.Draw(canvas)

    # ===== زخارف الزوايا =====
    corner_size = 22
    dc = p['deco_color']
    corners = [
        (inner + 4, inner + 4),
        (W - inner - 4 - corner_size,
inner + 4),
        (inner + 4, H - inner - 4 -
corner_size),
        (W - inner - 4 - corner_size, H
- inner - 4 - corner_size),
    ]
    for cx, cy in corners:
        draw.rectangle([cx, cy, cx +
corner_size, cy + corner_size], outline=dc,
width=2)

        draw.line(
            [(cx + corner_size // 2,
cy), (cx + corner_size // 2, cy +
corner_size)],
            fill=dc, width=1
        )
        draw.line(
            [(cx, cy + corner_size //
2), (cx + corner_size, cy + corner_size //
2)],
            fill=dc, width=1
        )

    #

```

```
# الشريط العلوي: ● خبر عاجل + شعار  
الصفحة  
#
```

```
font_badge = get_font(30)  
font_logo = get_font(44)  
  
# شارة خبر عاجل (يسار)  
badge_y = (BAR - 50) // 2 + 5  
_draw_breaking_news_badge(draw, 40,  
badge_y, font_badge, p.get('accent', (200,  
50, 50)))  
  
# شعار الصفحة (يمين)  
logo_radius = 32  
logo_cx = W - 70  
logo_cy = BAR // 2  
_draw_page_logo(draw, logo_cx,  
logo_cy, logo_radius, p, font_logo)  
  
# خط فاصل تحت الشريط العلوي  
draw.line(  
    [(inner + 30, BAR - 1), (W -  
inner - 30, BAR - 1)],  
    fill=p['deco_color'], width=1  
)  
  
#
```

```
# عنوان الخبر داخل الصورة
```



```

#
=====

font_title = get_font(40)
title_ar = ar(title)
max_title_w = W - PAD * 4
title_lines = wrap_text(title_ar,
font_title, max_title_w, draw)
if len(title_lines) > 3:
    title_lines = title_lines[:3]
    title_lines[-1] =
title_lines[-1][:30] + '...'

line_height = 56
total_h = len(title_lines) *
line_height
title_y_start = H - BAR - PAD -
total_h - 20

# خط زخرفي فوق العنوان
draw.line(
    [(ix + 100, title_y_start -
15), (W - ix - 100, title_y_start - 15)],
    fill=p['deco_color'], width=2
)

# رسم العنوان مع ظل
title_y = title_y_start
for line in title_lines:
    bbox = draw.textbbox((0, 0),
line, font=font_title)
    tw = bbox[2] - bbox[0]
    tx = W // 2 - tw // 2
    # ظل مزدوج

```

```

        for dx, dy in [(-2, -2), (2,
-2), (-2, 2), (2, 2)]:
            draw.text((tx + dx, title_y
+ dy), line, font=font_title, fill=(0, 0,
0))

            # نص أبيض ساطع
            draw.text((tx, title_y), line,
font=font_title, fill=(255, 255, 255))
            title_y += line_height

#
=====

=====

# الشريط السفلي: 17 التاريخ + اسم
الصفحة

#
=====

=====

font_date = get_font(26)
font_page = get_font(38)

# خط فاصل فوق الشريط السفلي
draw.line(
    [(inner + 30, H - BAR + 1), (W
- inner - 30, H - BAR + 1)],
    fill=p['deco_color'], width=1
)

# التاريخ بالعربي (يسار)
date_str = ar(format_arabic_date())
bbox = draw.textbbox((0, 0),
date_str, font=font_date)
date_h = bbox[3] - bbox[1]

```

```

# أيقونة تقويم
cal_size = 28
cal_x = 40
cal_y = H - BAR // 2 - cal_size //
2
_draw_calendar_icon(draw, cal_x,
cal_y, cal_size, p['bar_text'])

# نص التاريخ
date_x = cal_x + cal_size + 12
date_y = H - BAR // 2 - date_h // 2
- 6
draw.text((date_x, date_y),
date_str, font=font_date,
fill=p['bar_text'])

# اسم الصفحة (يمين) مع توجه
page_name_ar = ar(p['name'])
bbox = draw.textbbox((0, 0),
page_name_ar, font=font_page)
pn_w = bbox[2] - bbox[0]
pn_h = bbox[3] - bbox[1]

pn_x = W - 50 - pn_w
pn_y = H - BAR // 2 - pn_h // 2 - 6

# توجه
for dx, dy in [(-2, -2), (2, -2),
(-2, 2), (2, 2)]:
draw.text((pn_x + dx, pn_y +
dy), page_name_ar,
font=font_page,
fill=p['glow'])
# النص

```

```

        draw.text((pn_x, pn_y),
page_name_ar,
                    font=font_page,
fill=p['bar_text'])

        # خط مزخرف صغير بجانب الاسم
        draw.line(
            [(pn_x - 25, H - BAR // 2),
(pn_x - 8, H - BAR // 2)],
            fill=p['deco_color'], width=2
        )

        return canvas

    except Exception as e:
        logger.error(f"خطأ في إنشاء الصورة:
{e}")

    logger.debug(traceback.format_exc())
    return None

#
=====
=====
# النشر على فيسبوك
#
=====
=====
def post_to_facebook(news: Dict, page_key:
str) -> bool:
    """(محشن JPEG) نشر خبر على صفحة فيسبوك"""
    try:
        p = PAGES_CONFIG[page_key]

```

```

        token = p['token']
        page_id = p['page_id']

        if not token:
            logger.debug(f"لا يوجد توكن لـ {page_key}")
            return False

        image =
        create_post_image(news['title'],
        news.get('image_url', ''), p)

        if image:
            img_bytes = io.BytesIO()
            # JPEG بدل PNG: 70~ حجم أصغر %
            image.save(img_bytes,
            format='JPEG', quality=IMAGE_QUALITY,
            optimize=True)
            img_bytes.seek(0)

            resp = requests.post(

            f'https://graph.facebook.com/v18.0/{page_id
            }/photos',

                files={'source':
            ('post.jpg', img_bytes, 'image/jpeg')},
                data={'caption':
            news['title'], 'access_token': token},
                timeout=FB_POST_TIMEOUT,
            )
        else:
            # fallback: نص فقط
            resp = requests.post(

```

```

f'https://graph.facebook.com/v18.0/{page_id}
/feed',
        data={'message':
news['title'], 'access_token': token},
        timeout=FB_POST_TIMEOUT,
    )

    if resp.status_code in (200, 201):
        return True

    logger.warning(
        f"{page_key}: فيسبوك رفض النشر على"
    )

    f"{resp.status_code} -
{resp.text[:150]}"
    )
    return False

except requests.exceptions.Timeout:
    logger.warning(f"⌚ timeout عند النشر
على {page_key}")
    return False
except
requests.exceptions.RequestException as e:
    logger.warning(f"خطأ شبكة عند النشر على
{page_key}: {e}")
    return False
except Exception as e:
    logger.error(f"خطأ غير متوقع عند النشر
على {page_key}: {e}")
    return False

def post_to_all_pages(news: Dict) ->

```

```
List[str]:
    """نشر على جميع الصفحات النشطة بالتوازي"""
    posted: List[str] = []

    active_pages = [k for k, v in
PAGES_CONFIG.items() if v.get('token')]
    if not active_pages:
        logger.error("❌ لا توجد صفحات بتوكنات  
صالحة!")
        return posted

    def post_one(key: str) ->
Optional[str]:
        try:
            return key if
post_to_facebook(news, key) else None
        except Exception as e:
            logger.debug(f"خطأ في")
post_one({key}): {e}")
            return None

    with
ThreadPoolExecutor(max_workers=len(active_p
ages)) as executor:
        futures =
{executor.submit(post_one, k): k for k in
active_pages}
        try:
            for future in
as_completed(futures,
timeout=FB_POST_TIMEOUT * 2):
                try:
                    result =
future.result(timeout=1)
```

```

        if result:

posted.append(result)
            except Exception as e:
                logger.debug(f"خطأ في {e}")
future.result: {e}")
            except TimeoutError:
                logger.warning("⌚ timeout في
post_to_all_pages")
            except Exception as e:
                logger.warning(f"خطأ غير متوقع في {e}")
post_to_all_pages: {e}")

        return posted

#
=====
=====
# تحليل المعطيات من سطر الأوامر
#
=====
=====
def _parse_args() -> argparse.Namespace:
    """تحليل خيارات سطر الأوامر"""
    parser = argparse.ArgumentParser(
        description='نظام نشر الأخبار التلقائي على
        فيسبوك (v25)',

formatter_class=argparse.RawDescriptionHelp
Formatter,
        epilog=''
أمثلة:
%(prog)s                                     #

```



```

%(prog)s (VPS / PythonAnywhere) حلقة لا نهائية
%(prog)s --once #
%(prog)s (GitHub Actions) دورة واحدة وخروج
%(prog)s --once --max-posts 5 #
%(prog)s حد أقصى 5 منشورات في الدورة
%(prog)s --verify-sources #
%(prog)s فحص المصادر فقط بدون نشر
'''
.strip(),
)
parser.add_argument(
    '--once',
    action='store_true',
    help='مثالي لـ تنفيذ دورة واحدة وخروج (GitHub Actions)'
)
parser.add_argument(
    '--max-posts',
    type=int,
    default=10,
    metavar='N',
    help='--once حد أقصى للمنشورات في وضع (افتراضي: 10)'
)
parser.add_argument(
    '--verify-sources',
    action='store_true',
    help='فحص جميع مصادر الأخبار بدون نشر'
)
return parser.parse_args()

```

```

#
=====
=====

```

```

# وضع فحص المصادر (--verify-sources)
#
=====
=====
def _verify_sources_mode() -> int:
    """فحص جميع مصادر الأخبار وعرض حالة كل منها."""
    logger.info("🔍 وضع فحص المصادر - لن يتم النشر")
    logger.info("-" * 60)

    results: List[Tuple[str, bool, int, str]] = []

    def check_source(url: str) -> Tuple[str, bool, int, str]:
        try:
            resp = requests.get(url,
                                timeout=REQUEST_TIMEOUT,
                                headers=REQUEST_HEADERS)
            if resp.status_code != 200:
                return (url, False, 0,
                        f"HTTP {resp.status_code}")
            feed =
            feedparser.parse(resp.content)
            total = len(feed.entries)
            iraq_count = sum(
                1 for e in feed.entries
                if
                is_iraq_news(e.get('title', '') + ' ' +
                             e.get('summary', ''))
            )
            return (url, True, iraq_count,
                    f"{iraq_count}/{total} عراقية")
        except requests.exceptions.Timeout:

```

```

        return (url, False, 0,
"timeout")
        except Exception as e:
            return (url, False, 0, str(e)
[:50])

        with
ThreadPoolExecutor(max_workers=len(NEWS_SOU
RCES)) as executor:
            futures =
[executor.submit(check_source, src) for src
in NEWS_SOURCES]
            for future in as_completed(futures,
timeout=REQUEST_TIMEOUT * 3):
                try:

results.append(future.result(timeout=1))
                except Exception as e:
                    logger.debug(f"خطأ في فحص:
{e}")

            # ترتيب حسب النجاح
            results.sort(key=lambda x: (not x[1], -
x[2]))

            working = sum(1 for r in results if
r[1])
            total_iraq = sum(r[2] for r in results
if r[1])

            logger.info(f"\n🇮🇶 النتيجة:
{working}/{len(NEWS_SOURCES)} | مصدر يعمل "
f"{total_iraq} خبر عراقي
\nمرشح")

```

```

        for url, ok, count, msg in results:
            status = "✅" if ok else "❌"
            short_url = url.replace('https://',
            '').replace('http://', '')[0:50]
            logger.info(f" {status}
            {short_url:55} → {msg}")

        return 0 if working > 0 else 1

#
=====
=====
# وضع الدورة الواحدة (--once لـ GitHub Actions)
#
=====
=====
def _once_mode(max_posts: int,
active_pages: List[str]) -> int:
    """
    تنفيذ دورة واحدة كاملة ثم خروج. مصمّم لـ
    GitHub Actions."""
    logger.info(f"🔄 وضع الدورة الواحدة - حد
    {max_posts} منشور أقصى")
    start_time = time.time()

    # 1. جلب الأخبار الجديدة
    try:
        news_list = fetch_news()
        new_count = sum(
            1 for n in news_list
            if save_news(n['title'],
            n['url'], n.get('image_url', ''))
        )

```

```

        logger.info(f"📄 جلب {new_count} خبر  

        مرشح {len(news_list)} جديد من  

        except Exception as e:  

        logger.error(f"خطأ في جلب الأخبار:  

        {e}")

    # 2. خبر max_posts نشر حتى  

    post_count = 0  

    skip_count = 0  

    for i in range(max_posts):  

        if _shutdown_requested:  

            break

        unposted = get_unposted()  

        if not unposted:  

            logger.info("✓ لا توجد أخبار جديدة  

            للنشر")  

            break

        title_short = unposted['title']  

        [:50]  

        attempt_num =  

        unposted['retry_count'] + 1  

        logger.info(f"📝  

        [{i+1}/{max_posts}] نشر: {title_short}...  

        (محاولة {attempt_num})")

        try:  

            posted_pages =  

            post_to_all_pages(unposted)  

        except Exception as e:  

            logger.error(f"خطأ في النشر:  

            {e}")  

            posted_pages = []

```

```

        if posted_pages:
            mark_posted(unposted['id'])
            post_count += 1
            logger.info(
                f"✅ نُشر على {len(posted_pages)}/{len(active_pages)} صفحات "
                f"({', '.join(posted_pages)})"
            )
        else:
            new_retry = increment_retry(unposted['id'])
            if new_retry >= MAX_RETRIES_PER_NEWS:
                logger.warning(f"⚠️ تخطي بعد {MAX_RETRIES_PER_NEWS} محاولات")
                skip_count += 1
            else:
                logger.info(f"🔄 سيعاد لاحقاً {new_retry} محاولة")
                break # قد يكون فيسبوك معطل

# 3. تنظيف
try:
    deleted = cleanup_old()
    if deleted > 0:
        logger.info(f"🧹 حذف {deleted} خبر قديم")
except Exception as e:
    logger.debug(f"خطأ في التنظيف: {e}")

```

```

# 4. ملخص
elapsed = time.time() - start_time
logger.info("=" * 60)
logger.info(
    f"🇸🇦 | منشور {post_count} ملخص الدورة "
    f"{skip_count} تخطي | {elapsed:.1f} ثانية"
)
logger.info("=" * 60)

return 0

#
=====
=====
# افتراضي وضع الحلقة المستمرة -
VPS/PythonAnywhere)
#
=====
=====
def _continuous_mode(active_pages:
List[str]) -> int:
    """الحلقة المستمرة (الوضع الافتراضي)"""
    fetch_counter = 0
    cleanup_counter = 0
    post_count = 0
    start_time = time.time()

    while not _shutdown_requested:
        try:
            loop_start = time.time()

```

```

        # جلب دوري
        fetch_counter += 1
        if fetch_counter >=
FETCH_INTERVAL_CYCLES:
            fetch_counter = 0
            try:
                news_list =
fetch_news()

                new_count = sum(
                    1 for n in
news_list
                        if
save_news(n['title'], n['url'],
n.get('image_url', ''))
                )
                if new_count > 0:
                    logger.info(
                        f"📰 جلب
{new_count} خبر جديد من {len(news_list)}
مرشح"
                    )
            except Exception as e:
                logger.warning(f"خطأ في
جلب الأخبار: {e}")

        # نشر
        try:
            unposted = get_unposted()
            if unposted:
                title_short =
unposted['title'][:50]
                attempt_num =
unposted['retry_count'] + 1
                logger.info(f"📝 نشر:

```



```

{title_short}... (محاولة {attempt_num})")
        posted_pages =
post_to_all_pages(unposted)
        if posted_pages:

mark_posted(unposted['id'])
                post_count += 1
                logger.info(
                        f"🟢 نشر على
{len(posted_pages)}/{len(active_pages)}
صفحات "
                        f"({' ,
'.join(posted_pages)}) | إجمالي:
{post_count}"
                        )
                else:
                        new_retry =
increment_retry(unposted['id'])
                        if new_retry >=
MAX_RETRIES_PER_NEWS:
                                logger.warning(
                                        f"⚠️ تخطي
بعد {MAX_RETRIES_PER_NEWS} محاولات "
                                        )
                                else:

logger.info(f"🔄 سيعاد لاحقاً
{new_retry}")
                        else:
                                logger.debug("⌚ لا توجد
"أخبار جديدة")
                                except Exception as e:
                                        logger.warning(f"خطأ في دورة النشر: {e}")

```

```

        # تنظيف دوري
        cleanup_counter += 1
        if cleanup_counter >=
CLEANUP_INTERVAL_CYCLES:
            cleanup_counter = 0
            try:
                deleted = cleanup_old()
                elapsed_min =
(time.time() - start_time) / 60
                logger.info(
                    f"🧹 حذف {deleted}
| تشغيل: {elapsed_min:.0f}د | "
                    منشورات:
{post_count}"
                )
            except Exception as e:
                logger.debug(f"خطأ في
التنظيف: {e}")

        # انتظار مع استجابة للإشارات
        elapsed = time.time() -
loop_start
        wait = max(0.0,
LOOP_DELAY_SECONDS - elapsed)
        if wait > 0:
            end_wait = time.time() +
wait
            while time.time() <
end_wait and not _shutdown_requested:
                time.sleep(min(1.0,
end_wait - time.time()))

    except Exception as e:

```

```

        logger.error(f"خطأ غير متوقع في الحلقة: {e}")

    logger.debug(traceback.format_exc())
    time.sleep(5)
    continue

    elapsed_min = (time.time() -
start_time) / 60
    logger.info("=" * 60)
    logger.info("توقف النظام بنجاح")
    logger.info(f"الإحصائيات: {post_count} منشور في {elapsed_min:.0f} دقيقة")
    logger.info("=" * 60)
    return 0

#
=====
=====
# الدالة الرئيسية - dispatcher
#
=====
=====
def main() -> int:
    args = _parse_args()

    # Banner
    logger.info("=" * 60)
    logger.info("🚀 25 النسخة - النظام نشر الأخبار (GitHub Actions Ready)")
    if args.once:
        logger.info("📌 وضع: دورة واحدة (--once)")

```

```

elif args.verify_sources:
    logger.info("🔍 وضع: فحص المصادر (--verify-sources)")
else:
    logger.info("🔗 حلقة مستمرة (افتراضي)")
    logger.info(f"📰 المصادر: {len(NEWS_SOURCES)} مصدر إخباري")
    logger.info("=" * 60)

    # تسجيل معالجات الإشارات
    signal.signal(signal.SIGTERM, _signal_handler)
    signal.signal(signal.SIGINT, _signal_handler)

    # أو توكينات DB وضع فحص المصادر - لا يحتاج
    if args.verify_sources:
        return _verify_sources_mode()

    # تهيئة قاعدة البيانات
    db_ready = False
    for attempt in range(1, MAX_DB_INIT_ATTEMPTS + 1):
        if init_db():
            db_ready = True
            break
        logger.warning(f"⚠️ محاولة init_db {attempt}/{MAX_DB_INIT_ATTEMPTS}")
        if attempt < MAX_DB_INIT_ATTEMPTS:
            time.sleep(3)
    if not db_ready:
        logger.error("❌ فشلت تهيئة قاعدة البيانات، إيقاف")

```

```

        return 1

        # التحقق من توكنات الصفحات
        active_pages = [k for k, v in
PAGES_CONFIG.items() if v.get('token')]
        if not active_pages:
            logger.error("❌ لا توجد توكنات صالحة في
متغيرات البيئة")
            logger.error("اضبط واحد على الأقل
من:")
            logger.error("PAGE_SALSSAL,
PAGE_CHAI, PAGE_TABOGA, PAGE_TEIN")
            return 1
        logger.info(f"📄 صفحات مفغلة
({len(active_pages)}): {'',
''.join(active_pages)}")
        logger.info(f"🕒 بدء التشغيل:
{datetime.now():%Y-%m-%d %H:%M:%S}")

        # التشغيل بالوضع المطلوب
        if args.once:
            return _once_mode(args.max_posts,
active_pages)
        else:
            return
_continuous_mode(active_pages)

if __name__ == '__main__':
    sys.exit(main())

```